Semantic Space Abstractions for Partially Observable Deterministic Task and Motion Planning

Final Presentation

Hani Alassiri Alhabboub

23rd Jun, 2026

Chair of Machine Learning and Reasoning (Computer Science 6)

RWTH AACHEN UNIVERSITY

Roadmap

1. Introduction
2. Background
3. Related Work
4. Approach
5. Experimental Evaluation
6. Discussion

Introduction

- Goal: robots that carry out everyday tasks on their own — fetch an object, clear a table

Introduction

- Goal: robots that carry out everyday tasks on their own — fetch an object, clear a table
- This needs both multi-step reasoning *and* physical manipulation $\Rightarrow$ **Task and Motion Planning (TAMP)**

Introduction

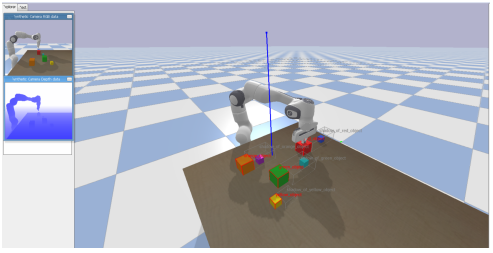
- Goal: robots that carry out everyday tasks on their own — fetch an object, clear a table
- This needs both multi-step reasoning *and* physical manipulation $\Rightarrow$ **Task and Motion Planning (TAMP)**
- TAMP links a high-level goal to feasible motion: a symbolic task plan, checked against geometry and kinematics

Introduction

- Goal: robots that carry out everyday tasks on their own — fetch an object, clear a table
- This needs both multi-step reasoning *and* physical manipulation $\Rightarrow$ **Task and Motion Planning (TAMP)**
- TAMP links a high-level goal to feasible motion: a symbolic task plan, checked against geometry and kinematics
- **But**: classical TAMP (such as PDDLStream) assumes a *fully observable*, *deterministic* world

Introduction: Planning under Occlusion

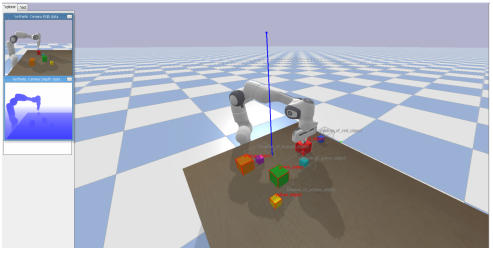
- Real scenes are **partially observable**: occlusion, limited sensor coverage



Target (cyan) hidden behind the green cube; one fixed camera (RGB/depth insets).

Introduction: Planning under Occlusion

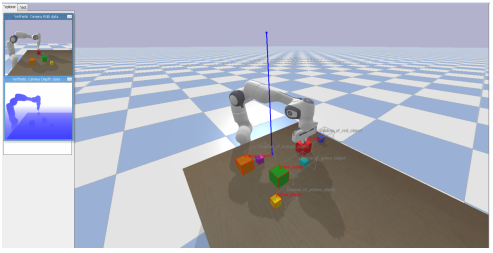
- Real scenes are **partially observable**: occlusion, limited sensor coverage
- The target can be hidden — the robot must *look* before it can act



Target (cyan) hidden behind the green cube; one fixed camera (RGB/depth insets).

Introduction: Planning under Occlusion

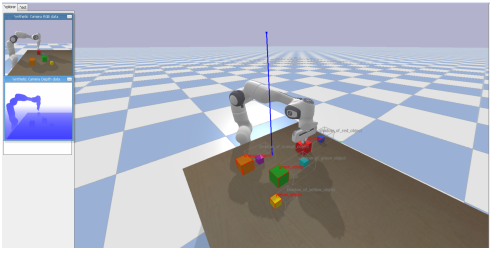
- Real scenes are **partially observable**: occlusion, limited sensor coverage
- The target can be hidden — the robot must *look* before it can act
- It needs a **belief** over where hidden objects are



Target (cyan) hidden behind the green cube; one fixed camera (RGB/depth insets).

Introduction: Planning under Occlusion

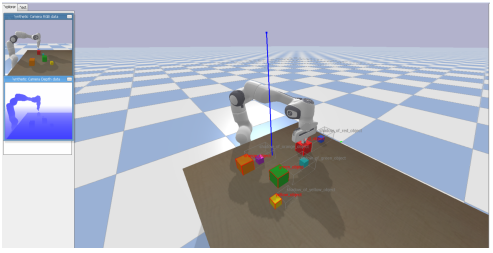
- Real scenes are **partially observable**: occlusion, limited sensor coverage
- The target can be hidden — the robot must *look* before it can act
- It needs a **belief** over where hidden objects are
- A uniform voxel grid is the obvious way to represent it — but it **blows up** $\Rightarrow$ too slow to plan



Target (cyan) hidden behind the green cube; one fixed camera (RGB/depth insets).

Introduction: Planning under Occlusion

- Real scenes are **partially observable**: occlusion, limited sensor coverage
- The target can be hidden — the robot must *look* before it can act
- It needs a **belief** over where hidden objects are
- A uniform voxel grid is the obvious way to represent it — but it **blows up** ⇒ too slow to plan

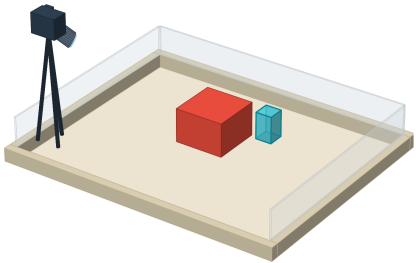


Target (cyan) hidden behind the green cube; one fixed camera (RGB/depth insets).

Research question: How can we define and integrate a belief representation based on *semantic partitioning* into a partially observable deterministic (POD) TAMP framework?

Introduction: The Boxel Idea

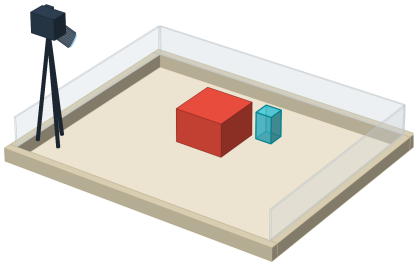
- **Idea**: put the structure of space into the symbolic planning state



Full construction in the Approach section.

Introduction: The Boxel Idea

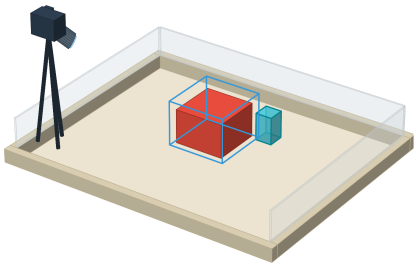
- **Idea**: put the structure of space into the symbolic planning state
- The **Boxel** — a *box-shaped cell* of the workspace that either



Full construction in the Approach section.

Introduction: The Boxel Idea

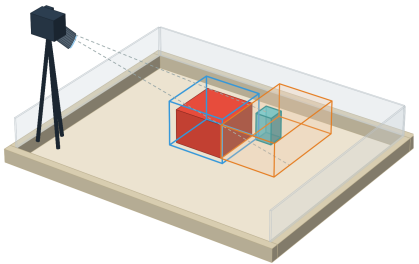
- **Idea**: put the structure of space into the symbolic planning state
- The **Boxel** — a *box-shaped cell* of the workspace that either
  - bounds a detected object,



Full construction in the Approach section.

Introduction: The Boxel Idea

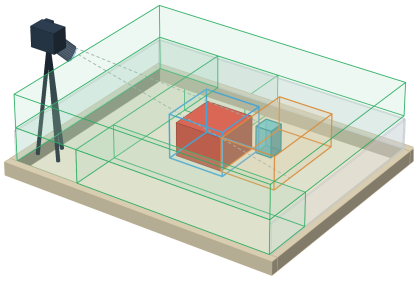
- **Idea**: put the structure of space into the symbolic planning state
- The **Boxel** — a *box-shaped cell* of the workspace that either
  - bounds a detected object,
  - marks the region it *occludes* (shadow), or



Full construction in the Approach section.

Introduction: The Boxel Idea

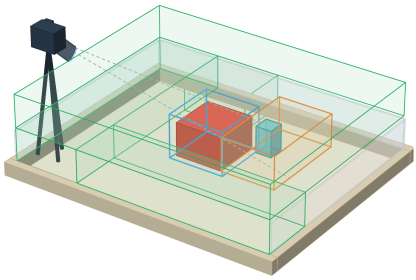
- **Idea**: put the structure of space into the symbolic planning state
- The **Boxel** — a *box-shaped cell* of the workspace that either
  - bounds a detected object,
  - marks the region it *occludes* (shadow), or
  - covers free space



Full construction in the Approach section.

Introduction: The Boxel Idea

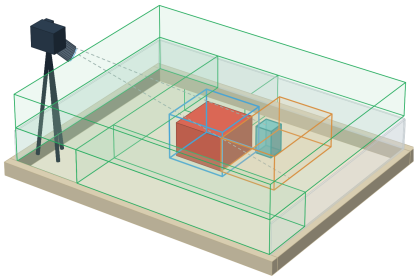
- **Idea**: put the structure of space into the symbolic planning state
- The **Boxel** — a *box-shaped cell* of the workspace that either
  - bounds a detected object,
  - marks the region it *occludes* (shadow), or
  - covers free space
- Fine where it matters, coarse elsewhere — a handful of cells, not a dense grid



Full construction in the Approach section.

Introduction: The Boxel Idea

- **Idea**: put the structure of space into the symbolic planning state
- The **Boxel** — a *box-shaped cell* of the workspace that either
  - bounds a detected object,
  - marks the region it *occludes* (shadow), or
  - covers free space
- Fine where it matters, coarse elsewhere — a handful of cells, not a dense grid
- A classical planner reasons over these few cells, and *looks* only where it must



Full construction in the Approach section.

Background: Classical Planning

- A planning problem P: reach a **goal** from an **initial state** by choosing **actions** — captured by its **state model** $S(P)$

Background: Classical Planning

- A planning problem P: reach a **goal** from an **initial state** by choosing **actions** — captured by its **state model** $S(P)$

$$S(P) = \langle S,\ s_0,\ S_G,\ Act,\ A,\ f \rangle$$

Background: Classical Planning

- A planning problem P: reach a **goal** from an **initial state** by choosing **actions** — captured by its **state model** $S(P)$

$$S(P) = \langle S,\ s_0,\ S_G,\ Act,\ A,\ f \rangle$$

states S

Background: Classical Planning

- A planning problem P: reach a **goal** from an **initial state** by choosing **actions** — captured by its **state model** $S(P)$

$$S(P) = \langle S,\ s_0,\ S_G,\ Act,\ A,\ f \rangle$$

states S; initial state s_0

Background: Classical Planning

- A planning problem P: reach a **goal** from an **initial state** by choosing **actions** — captured by its **state model** $S(P)$

$$S(P) = \langle S,\ s_0,\ S_G,\ Act,\ A,\ f \rangle$$

states S; initial state s_0; goal states $S_G \subseteq S$

Background: Classical Planning

- A planning problem P: reach a **goal** from an **initial state** by choosing **actions** — captured by its **state model** $S(P)$

$$S(P) \;=\; \langle\, S,\; s_0,\; S_G,\; Act,\; A,\; f \,\rangle$$

states S; initial state s_0; goal states $S_G \subseteq S$; actions Act

Background: Classical Planning

- A planning problem P: reach a **goal** from an **initial state** by choosing **actions** — captured by its **state model** $S(P)$

$$S(P) = \langle S,\ s_0,\ S_G,\ Act,\ A,\ f \rangle$$

states S; initial state s_0; goal states $S_G \subseteq S$; actions Act; $A(s)$, the actions applicable in s

Background: Classical Planning

- A planning problem P: reach a **goal** from an **initial state** by choosing **actions** — captured by its **state model** $S(P)$

$$S(P) \;=\; \langle\, S,\; s_0,\; S_G,\; Act,\; A,\; f \,\rangle$$

states S; initial state s_0; goal states $S_G \subseteq S$; actions Act; $A(s)$, the actions applicable in s; and the deterministic transition $s' = f(a, s)$.

Background: Classical Planning

- A planning problem P: reach a **goal** from an **initial state** by choosing **actions** — captured by its **state model** $S(P)$

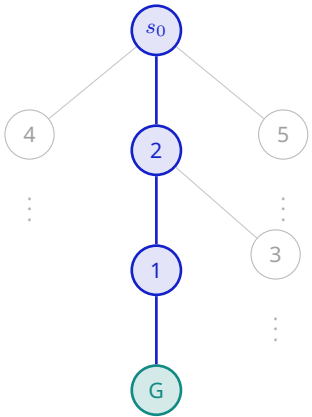
$$S(P) \;=\; \langle\, S,\; s_0,\; S_G,\; Act,\; A,\; f \,\rangle$$

states S; initial state s_0; goal states $S_G \subseteq S$; actions Act; $A(s)$, the actions applicable in s; and the deterministic transition $s' = f(a, s)$.

- **STRIPS / PDDL** represents it compactly: a *factored* state is the set of true atoms, e.g. $\{$`(on A B)`, `(handempty)`$\}$; an action has preconditions and add / delete effects

Background: Solving It — Heuristic Search

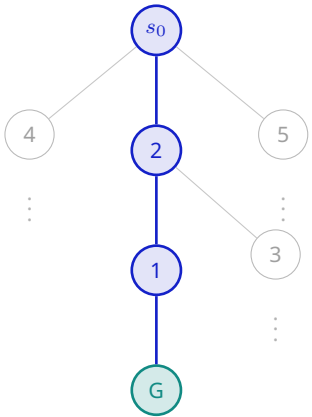
- Expand states **best-first**, ordered by a *heuristic* $h(s)$ — an estimate of the remaining distance to the goal



Labels: heuristic h. Lowest-h path in blue; grey branches are never expanded.

Background: Solving It — Heuristic Search

- Expand states **best-first**, ordered by a *heuristic* $h(s)$ — an estimate of the remaining distance to the goal
- Promising states go first, so the goal is reached **without enumerating** the whole state space



Labels: heuristic h. Lowest-h path in blue; grey branches are never expanded.

Background: When the World Is Not Fully Known

- Unlike classical planning, a robot rarely knows the **exact state**: it cannot see *behind* an object, so it cannot tell where a hidden object is — **partial observability**

Background: When the World Is Not Fully Known

- Unlike classical planning, a robot rarely knows the **exact state**: it cannot see *behind* an object, so it cannot tell where a hidden object is — **partial observability**
- So it holds a **belief**: the set of world states still consistent with what it has seen; **sensing** shrinks it

Background: When the World Is Not Fully Known

- Unlike classical planning, a robot rarely knows the **exact state**: it cannot see *behind* an object, so it cannot tell where a hidden object is — **partial observability**
- So it holds a **belief**: the set of world states still consistent with what it has seen; **sensing** shrinks it

Example — a target hidden behind one of three objects:

Background: Compiling Knowledge into Classical Planning

- A **value fluent** `obj_at_boxel(cup,B)` alone is ambiguous: a bare `(not ...)` can't tell "looked, it's empty" from "never looked"

Background: Compiling Knowledge into Classical Planning

- A **value fluent** `obj_at_boxel(cup,B)` alone is ambiguous: a bare `(not ...)` can't tell "looked, it's empty" from "never looked"
- A **Know-If fluent** `obj_at_boxel_KIF(cup,B)` adds the missing bit — *is the value known yet?* (Petrick & Bacchus [Pet+02])

Background: Compiling Knowledge into Classical Planning

- A **value fluent** `obj_at_boxel(cup,B)` alone is ambiguous: a bare `(not ...)` can't tell "looked, it's empty" from "never looked"
- A **Know-If fluent** `obj_at_boxel_KIF(cup,B)` adds the missing bit — *is the value known yet?* (Petrick & Bacchus [Pet+02])

Example — "is the cup in region B?"

before sensing B

```
obj_at_boxel(cup,B)
= ?  (don't read it yet)
```

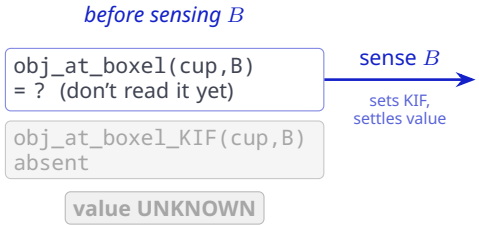
```
obj_at_boxel_KIF(cup,B)
absent
```

value UNKNOWN

Background: Compiling Knowledge into Classical Planning

- A **value fluent** `obj_at_boxel(cup,B)` alone is ambiguous: a bare `(not ...)` can't tell "looked, it's empty" from "never looked"
- A **Know-If fluent** `obj_at_boxel_KIF(cup,B)` adds the missing bit — *is the value known yet?* (Petrick & Bacchus [Pet+02])

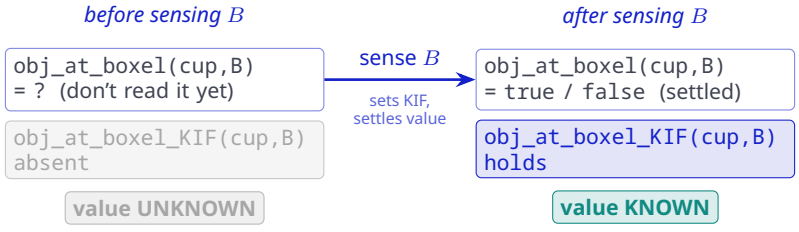
Example — "is the cup in region B?"



Background: Compiling Knowledge into Classical Planning

- A **value fluent** `obj_at_boxel(cup,B)` alone is ambiguous: a bare `(not ...)` can't tell "looked, it's empty" from "never looked"
- A **Know-If fluent** `obj_at_boxel_KIF(cup,B)` adds the missing bit — *is the value known yet?* (Petrick & Bacchus [Pet+02])

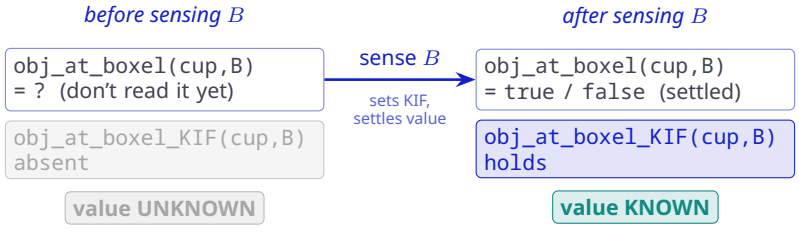
Example — "is the cup in region B?"



Background: Compiling Knowledge into Classical Planning

- A **value fluent** `obj_at_boxel(cup,B)` alone is ambiguous: a bare `(not ...)` can't tell "looked, it's empty" from "never looked"
- A **Know-If fluent** `obj_at_boxel_KIF(cup,B)` adds the missing bit — *is the value known yet?* (Petrick & Bacchus [Pet+02])

Example — "is the cup in region B?"



This **compiles** partial observability into a **classical** problem: the planner now seeks a *state of knowledge* — the target's region becomes *known* (Geffner & Bonet [Gef+13]).

Background: What Kind of Uncertainty?

Two very different reasons a plan can be uncertain:

- **Random outcomes** — e.g. a move that occasionally slips sideways. This needs a **POMDP**: a probability distribution over states and a *policy* — expressive, but computationally very demanding (Kaelbling et al. [Kae+98])

Background: What Kind of Uncertainty?

Two very different reasons a plan can be uncertain:

- **Random outcomes** — e.g. a move that occasionally slips sideways. This needs a **POMDP**: a probability distribution over states and a *policy* — expressive, but computationally very demanding (Kaelbling et al. [Kae+98])
- **Missing knowledge** — the world is fixed and predictable; the robot simply cannot *see* where the target is. This needs only to track what is **known vs. unknown** — partially observable *deterministic* (POD) planning (Geffner & Bonet [Gef+13])

Background: What Kind of Uncertainty?

Two very different reasons a plan can be uncertain:

- **Random outcomes** — e.g. a move that occasionally slips sideways. This needs a **POMDP**: a probability distribution over states and a *policy* — expressive, but computationally very demanding (Kaelbling et al. [Kae+98])
- **Missing knowledge** — the world is fixed and predictable; the robot simply cannot *see* where the target is. This needs only to track what is **known vs. unknown** — partially observable *deterministic* (POD) planning (Geffner & Bonet [Gef+13])

Ours is the second kind: the robot's actions are reliable — the only uncertainty is occlusion. So no probabilities; we track knowledge, and sensing resolves it.

Background: TAMP and PDDLStream

- So far we have planned over *discrete* symbols — but every action also needs *feasible motion*: TAMP's *symbolic* $\leftrightarrow$ *continuous* gap

Background: TAMP and PDDLStream

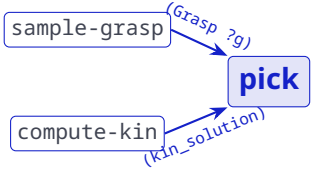
- So far we have planned over *discrete* symbols — but every action also needs *feasible motion*: TAMP's *symbolic* $\leftrightarrow$ *continuous* gap
- **PDDLStream** adds **streams**: samplers called *on demand* that produce continuous values (a grasp, an IK configuration, a path) and **certify** them as PDDL facts (Garrett et al. [Gar+18])

Background: TAMP and PDDLStream

- So far we have planned over *discrete* symbols — but every action also needs *feasible motion*: TAMP's *symbolic* ↔ *continuous* gap
- **PDDLStream** adds **streams**: samplers called *on demand* that produce continuous values (a grasp, an IK configuration, a path) and **certify** them as PDDL facts (Garrett et al. [Gar+18])

Example — our `pick` action:

```
(:action pick :parameters (?o ?b ?g ?q)
 :precondition (and (handempty)
    (obj_at_boxel ?o ?b)
    (at_config ?q)                    // from move
    (Grasp ?g)                        // sample-grasp
    (kin_solution ?o ?b ?g ?q)))
 :effect (and (holding ?o) ...))
```



Each stream certifies one blue fact `pick` needs. The trajectory stream `plan-motion` feeds the preceding `move`, which sets `at_config`.

Related Work: TAMP under Partial Observability

- **POMDP-based** (TAMPURA): learns an abstract MDP and solves it with LAO* — proactive, but probabilistic and sub-symbolic (Curtis et al. [Cur+24])

Related Work: TAMP under Partial Observability

- **POMDP-based** (TAMPURA): learns an abstract MDP and solves it with LAO* — proactive, but probabilistic and sub-symbolic (Curtis et al. [Cur+24])
- **Reactive, partially-grounded plans**: defer uncertainty to execution; no probabilities, but cannot plan multi-step sensing (Pan et al. [Pan+24])

Related Work: TAMP under Partial Observability

- **POMDP-based** (TAMPURA): learns an abstract MDP and solves it with LAO* — proactive, but probabilistic and sub-symbolic (Curtis et al. [Cur+24])
- **Reactive, partially-grounded plans**: defer uncertainty to execution; no probabilities, but cannot plan multi-step sensing (Pan et al. [Pan+24])
- **Belief from foundation models**: a VLM / LLM estimates where objects are (Zhao et al. [Zha+25])

Related Work: TAMP under Partial Observability

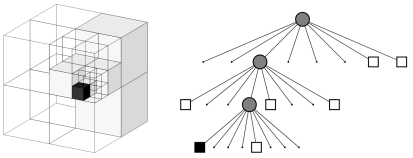
- **POMDP-based** (TAMPURA): learns an abstract MDP and solves it with LAO* — proactive, but probabilistic and sub-symbolic (Curtis et al. [Cur+24])
- **Reactive, partially-grounded plans**: defer uncertainty to execution; no probabilities, but cannot plan multi-step sensing (Pan et al. [Pan+24])
- **Belief from foundation models**: a VLM / LLM estimates where objects are (Zhao et al. [Zha+25])
- **Determinize-and-replan**: SS-Replan replans deterministically in belief space on each observation (Garrett et al. [Gar+20]) — we *share* this. The POD **K-replanner** compiles partial observability to classical via knowledge literals (Bonet & Geffner [Bon+11]), but is pure task planning; we bring that line *into TAMP*, on named **Boxel** volumes

Related Work: Spatial Representations under Occlusion

What we look for:

occlusion as *named, symbolic state* a planner can resolve.

- **Octree maps** (OctoMap): map *where matter is*; unobserved space is only absent evidence, and subdivision is geometric, not semantic (Hornung et al. [Hor+13])



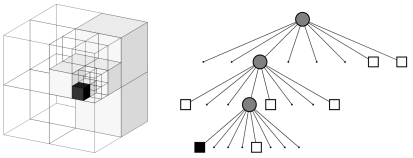
An octree map: free (white) and occupied (black) cells, with its tree (after Hornung et al.).

Related Work: Spatial Representations under Occlusion

What we look for:

occlusion as *named, symbolic state* a planner can resolve.

- **Octree maps** (OctoMap): map *where matter is*; unobserved space is only absent evidence, and subdivision is geometric, not semantic (Hornung et al. [Hor+13])
- **Learned critical regions**: abstract space by *task relevance*, but predicted once per scene and carry no occlusion (Molina et al. [Mol+20])



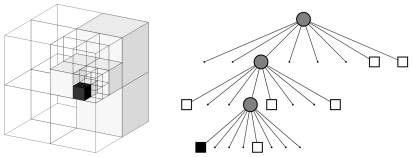
An octree map: free (white) and occupied (black) cells, with its tree (after Hornung et al.).

Related Work: Spatial Representations under Occlusion

What we look for:

occlusion as *named, symbolic state* a planner can resolve.

- **Octree maps** (OctoMap): map *where matter is*; unobserved space is only absent evidence, and subdivision is geometric, not semantic (Hornung et al. [Hor+13])
- **Learned critical regions**: abstract space by *task relevance*, but predicted once per scene and carry no occlusion (Molina et al. [Mol+20])
- **Occlusion-aware planners**: keep occlusion in a *probabilistic belief* (visibility grid or pose distribution), not a named volume (Garrett et al. [Gar+20])



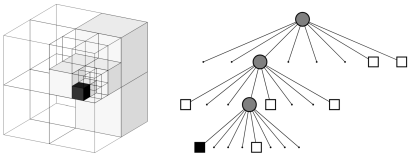
An octree map: free (white) and occupied (black) cells, with its tree (after Hornung et al.).

Related Work: Spatial Representations under Occlusion

What we look for:

occlusion as *named, symbolic state* a planner can resolve.

- **Octree maps** (OctoMap): map *where matter is*; unobserved space is only absent evidence, and subdivision is geometric, not semantic (Hornung et al. [Hor+13])
- **Learned critical regions**: abstract space by *task relevance*, but predicted once per scene and carry no occlusion (Molina et al. [Mol+20])
- **Occlusion-aware planners**: keep occlusion in a *probabilistic belief* (visibility grid or pose distribution), not a named volume (Garrett et al. [Gar+20])



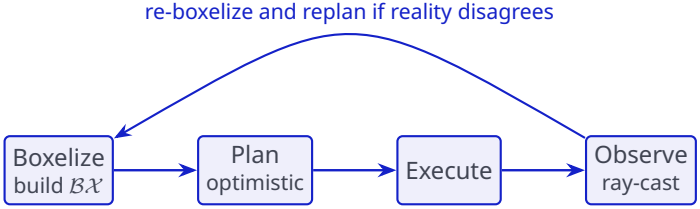
An octree map: free (white) and occupied (black) cells, with its tree (after Hornung et al.).

The gap: none names the occluded *volume* as symbolic state that sensing resolves inside a PDDLStream TAMP loop — what this thesis fills.

Approach: The Sense–Plan–Act Loop

The method, end to end:

boxelize the scene, plan optimistically, execute, observe — then loop. The rest of this section fills in each step.

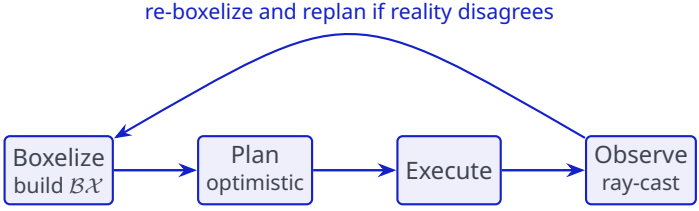


- Plan optimistically, execute, **repair by replanning**

Approach: The Sense–Plan–Act Loop

The method, end to end:

boxelize the scene, plan optimistically, execute, observe — then loop. The rest of this section fills in each step.

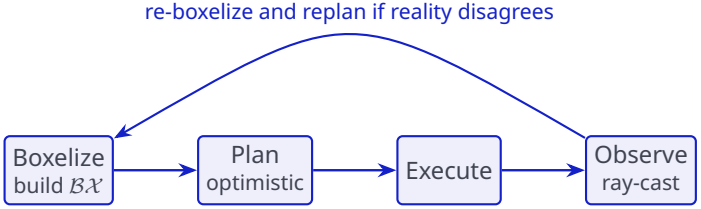


- Plan optimistically, execute, **repair by replanning**
- **Re-boxelize** when a placement consumes a cell or a new occluder appears

Approach: The Sense–Plan–Act Loop

The method, end to end:

boxelize the scene, plan optimistically, execute, observe — then loop. The rest of this section fills in each step.

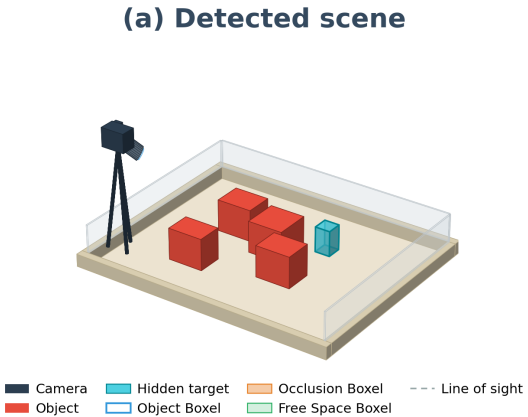


- Plan optimistically, execute, **repair by replanning**
- **Re-boxelize** when a placement consumes a cell or a new occluder appears
- **Time-bounded**: ends on goal, no plan, search exhausted, or budget

Approach: Building Boxels

Start from the objects, the camera, and the workspace — no Boxels yet

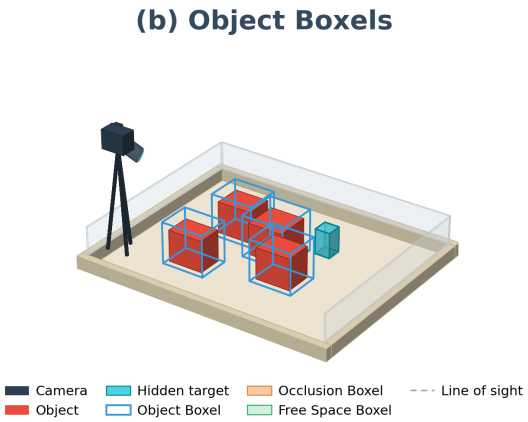
1. **Object Boxels**
   wrap each object in a tight box
2. **Shadow Boxels**
   add the region each object hides from the camera
3. **Free-space Boxels**
   begin with the whole workspace as one cell
   a cell that is empty becomes a free Boxel; otherwise cut it into 8 smaller cells, stopping at a minimum size
4. **Merge** free Boxels that share a face



Approach: Building Boxels

Start from the objects, the camera, and the workspace — no Boxels yet

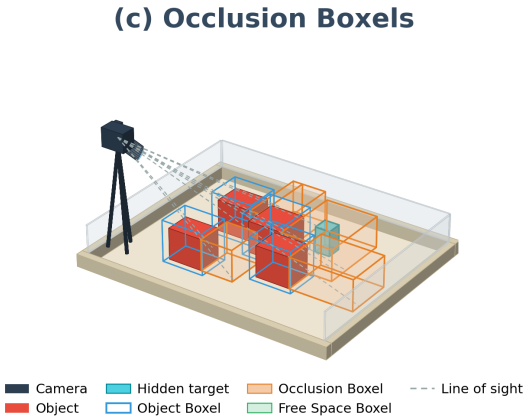
1. **Object Boxels**
   wrap each object in a tight box
2. **Shadow Boxels**
   add the region each object hides from the camera
3. **Free-space Boxels**
   begin with the whole workspace as one cell
   a cell that is empty becomes a free Boxel; otherwise cut it into 8 smaller cells, stopping at a minimum size
4. **Merge** free Boxels that share a face



Approach: Building Boxels

Start from the objects, the camera, and the workspace — no Boxels yet

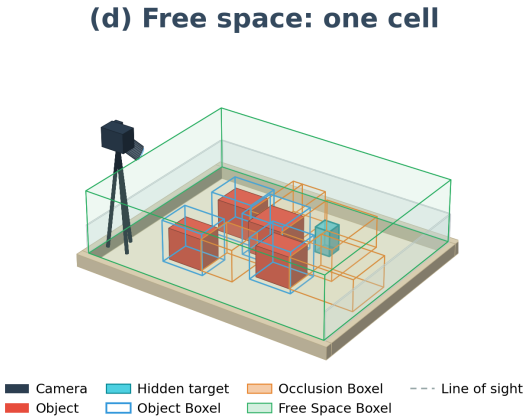
1. **Object Boxels**
   wrap each object in a tight box
2. **Shadow Boxels**
   add the region each object hides from the camera
3. **Free-space Boxels**
   begin with the whole workspace as one cell
   a cell that is empty becomes a free Boxel; otherwise cut it into 8 smaller cells, stopping at a minimum size
4. **Merge** free Boxels that share a face



Approach: Building Boxels

Start from the objects, the camera, and the workspace — no Boxels yet

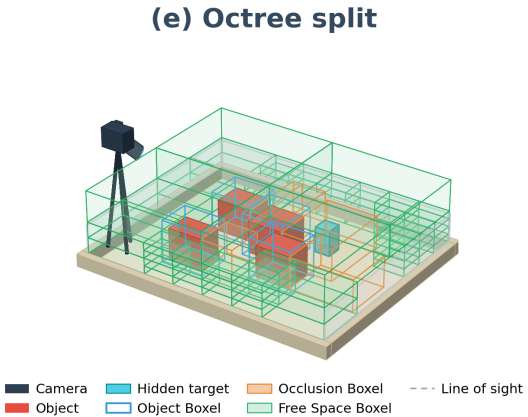
1. **Object Boxels**
   wrap each object in a tight box
2. **Shadow Boxels**
   add the region each object hides from the camera
3. **Free-space Boxels**
   begin with the whole workspace as one cell
   a cell that is empty becomes a free Boxel; otherwise cut it into 8 smaller cells, stopping at a minimum size
4. **Merge** free Boxels that share a face



Approach: Building Boxels

Start from the objects, the camera, and the workspace — no Boxels yet

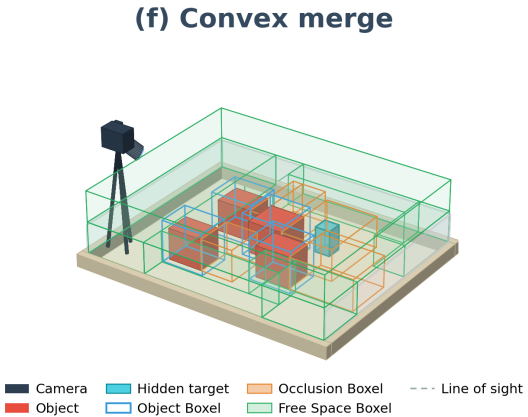
1. **Object Boxels**
   wrap each object in a tight box
2. **Shadow Boxels**
   add the region each object hides from the camera
3. **Free-space Boxels**
   begin with the whole workspace as one cell
   a cell that is empty becomes a free Boxel; otherwise cut it into 8 smaller cells, stopping at a minimum size
4. **Merge** free Boxels that share a face



Approach: Building Boxels

Start from the objects, the camera, and the workspace — no Boxels yet

1. **Object Boxels**
   wrap each object in a tight box
2. **Shadow Boxels**
   add the region each object hides from the camera
3. **Free-space Boxels**
   begin with the whole workspace as one cell
   a cell that is empty becomes a free Boxel; otherwise cut it into 8 smaller cells, stopping at a minimum size
4. **Merge** free Boxels that share a face



Approach: Belief over Boxels

- The Boxel partition *is* the belief support: a hidden target gives **one candidate world per Boxel** it could occupy ($\mathcal{S}_0$)

Example: target t behind one of 3 occluders

belief $\mathcal{S}_0$: 3 worlds

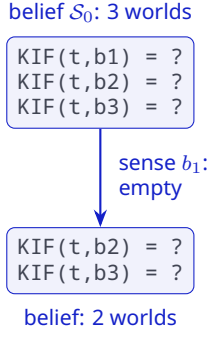
```
KIF(t,b1) = ?
KIF(t,b2) = ?
KIF(t,b3) = ?
```

$\texttt{KIF}(t, b_i)$: is target t in shadow Boxel b_i? The unknown ($?$) ones are the candidate worlds.

Approach: Belief over Boxels

- The Boxel partition *is* the belief support: a hidden target gives **one candidate world per Boxel** it could occupy ($\mathcal{S}_0$)
- **Sensing** a Boxel rules it in or out, so the candidate set **shrinks** as the robot looks — no probabilities

Example: target t behind one of 3 occluders

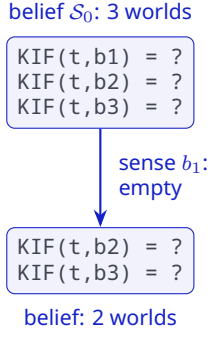


$\texttt{KIF}(t, b_i)$: is target t in shadow Boxel b_i? The unknown ($?$) ones are the candidate worlds.

Approach: Belief over Boxels

- The Boxel partition *is* the belief support: a hidden target gives **one candidate world per Boxel** it could occupy ($\mathcal{S}_0$)
- **Sensing** a Boxel rules it in or out, so the candidate set **shrinks** as the robot looks — no probabilities
- Carried by **Know-If fluents**, so a plain classical planner reasons over the hidden object — **no separate belief solver**

Example: target t behind one of 3 occluders



$\texttt{KIF}(t, b_i)$: is target t in shadow Boxel b_i? The unknown ($?$) ones are the candidate worlds.

Approach: PDDLStream Realization

The sensing action (optimistic, calls no streams)

```
sense(?o, ?region)
  pre:  view_clear(?region) ∧ ¬obj_at_boxel_KIF(?o, ?region)
  eff:  obj_at_boxel_KIF(?o, ?region), obj_at_boxel(?o, ?region)     // found
```

- A fixed camera observes the scene, so `sense` needs only a clear line of sight — no sensing pose to sample

Approach: PDDLStream Realization

The sensing action (optimistic, calls no streams)

```
sense(?o, ?region)
  pre:  view_clear(?region) ∧ ¬obj_at_boxel_KIF(?o, ?region)
  eff:  obj_at_boxel_KIF(?o, ?region), obj_at_boxel(?o, ?region)     // found
```

- A fixed camera observes the scene, so `sense` needs only a clear line of sight — no sensing pose to sample
- Four **streams** certify the continuous facts manipulation needs:

Approach: PDDLStream Realization

The sensing action (optimistic, calls no streams)

```
sense(?o, ?region)
  pre:  view_clear(?region) ∧ ¬obj_at_boxel_KIF(?o, ?region)
  eff:  obj_at_boxel_KIF(?o, ?region), obj_at_boxel(?o, ?region)     // found
```

- A fixed camera observes the scene, so `sense` needs only a clear line of sight — no sensing pose to sample
- Four **streams** certify the continuous facts manipulation needs:
  - `sample-grasp` — a top-down grasp

Approach: PDDLStream Realization

The sensing action (optimistic, calls no streams)

```
sense(?o, ?region)
  pre:  view_clear(?region) ∧ ¬obj_at_boxel_KIF(?o, ?region)
  eff:  obj_at_boxel_KIF(?o, ?region), obj_at_boxel(?o, ?region)     // found
```

- A fixed camera observes the scene, so `sense` needs only a clear line of sight — no sensing pose to sample
- Four **streams** certify the continuous facts manipulation needs:
  - `sample-grasp` — a top-down grasp
  - `compute-kin` — an IK configuration to pick or place at a Boxel

Approach: PDDLStream Realization

The sensing action (optimistic, calls no streams)

```
sense(?o, ?region)
  pre:  view_clear(?region) ∧ ¬obj_at_boxel_KIF(?o, ?region)
  eff:  obj_at_boxel_KIF(?o, ?region), obj_at_boxel(?o, ?region)     // found
```

- A fixed camera observes the scene, so `sense` needs only a clear line of sight — no sensing pose to sample
- Four **streams** certify the continuous facts manipulation needs:
  - `sample-grasp` — a top-down grasp
  - `compute-kin` — an IK configuration to pick or place at a Boxel
  - `compute-stack-kin` — an IK configuration to stack on an object

Approach: PDDLStream Realization

The sensing action (optimistic, calls no streams)

```
sense(?o, ?region)
  pre:  view_clear(?region) ∧ ¬obj_at_boxel_KIF(?o, ?region)
  eff:  obj_at_boxel_KIF(?o, ?region), obj_at_boxel(?o, ?region)     // found
```

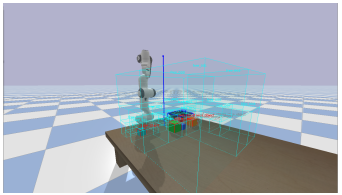
- A fixed camera observes the scene, so `sense` needs only a clear line of sight — no sensing pose to sample
- Four **streams** certify the continuous facts manipulation needs:
  - `sample-grasp` — a top-down grasp
  - `compute-kin` — an IK configuration to pick or place at a Boxel
  - `compute-stack-kin` — an IK configuration to stack on an object
  - `plan-motion` — a collision-free trajectory (RRT-Connect (Kuffner & LaValle [Kuf+00]))

Experimental Evaluation: Questions & Setup

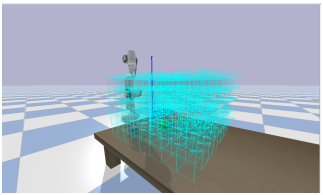
- **RQ1 (ablation):** does a task-relevant *semantic* partition cut planning cost and raise success against a *uniform* grid at the same cell scale?

Experimental Evaluation: Questions & Setup

- **RQ1 (ablation):** does a task-relevant *semantic* partition cut planning cost and raise success against a *uniform* grid at the same cell scale?



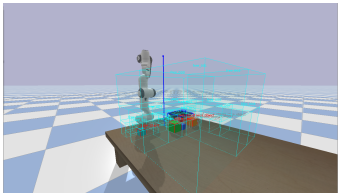
semantic



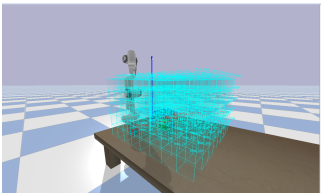
uniform

Experimental Evaluation: Questions & Setup

- **RQ1 (ablation):** does a task-relevant *semantic* partition cut planning cost and raise success against a *uniform* grid at the same cell scale?



semantic

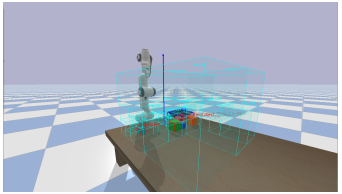


uniform

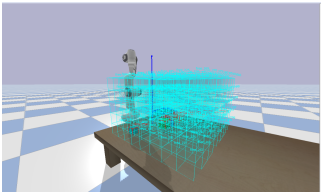
- **RQ2 (comparison):** how does deterministic POD-TAMP compare to a POMDP-based baseline (TAMPURA (Curtis et al. [Cur+24])) on the closest hidden-object task?

Experimental Evaluation: Questions & Setup

- **RQ1 (ablation):** does a task-relevant *semantic* partition cut planning cost and raise success against a *uniform* grid at the same cell scale?



semantic

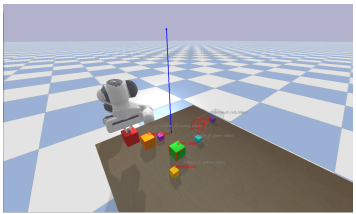


uniform

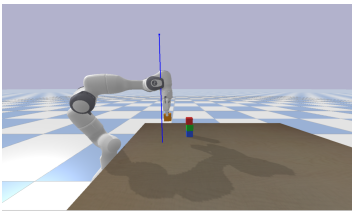
- **RQ2 (comparison):** how does deterministic POD-TAMP compare to a POMDP-based baseline (TAMPURA (Curtis et al. [Cur+24])) on the closest hidden-object task?

Setup: PyBullet (Coumans & Bai [Cou+21]) + 7-DOF Franka Panda, one fixed camera, **oracle** perception; 3 goals $\times$ 3 difficulty levels $\times$ 30 seeds, 1800 s budget each. Oracle perception isolates the representation — an ablation, not a benchmark win.

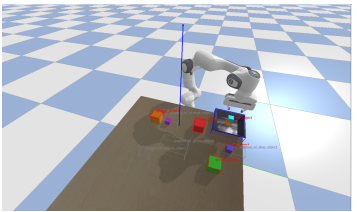
Experimental Evaluation: Tasks



`holding`: retrieve a hidden target

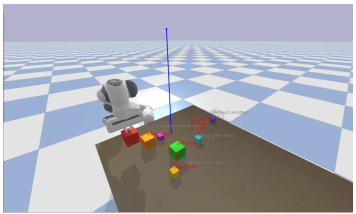


`stack`: build a tower to height h

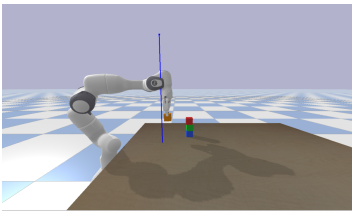


`find-and-tray-stack`: find, then stack on a tray

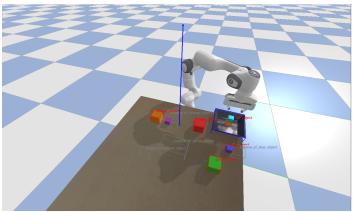
Experimental Evaluation: Tasks



`holding`: retrieve a hidden target



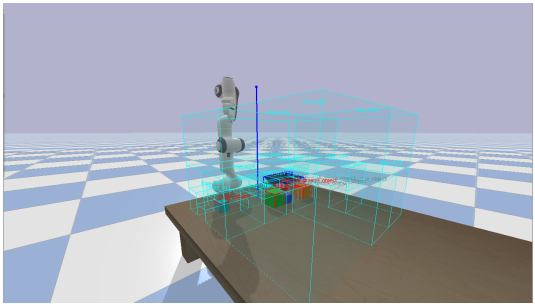
`stack`: build a tower to height h



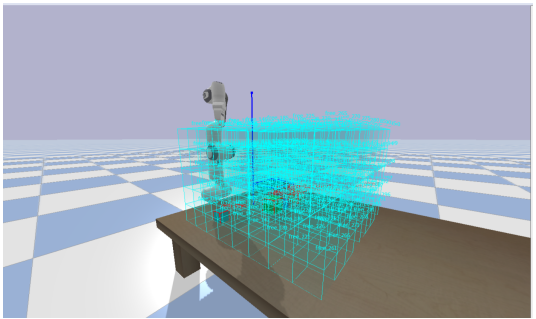
`find-and-tray-stack`: find, then stack on a tray

- Difficulty axis: number of occluders $n_{\text{occ}} \in \{2, 3, 4\}$ (search goals), or stack height $h \in \{2, 3, 4\}$ — each adds shadow regions or tower layers

Results: Semantic vs Uniform Partition



semantic: ≈22–38 task-relevant Boxels



uniform: ∼300 fixed cells over the whole workspace

Same cell size, same planner — the adaptive partition grounds and searches an order of magnitude fewer cells.

Results: Success and Planning Time

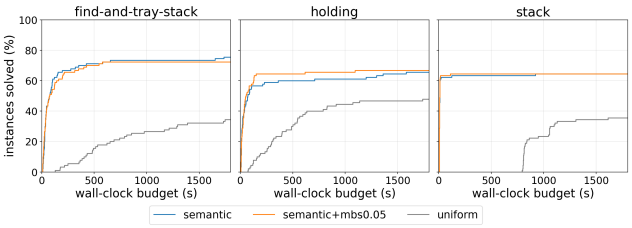
goal	variant	success	mean plan time (s)
find-and-tray-stack	**semantic**	**75.6 %**	**124.6**
	uniform	34.4 %	692.1
holding	**semantic**	**65.6 %**	**134.3**
	uniform	47.8 %	438.5
stack	**semantic**	**64.4 %**	**17.2**
	uniform	35.6 %	924.1

Results: Success and Planning Time

goal	variant	success	mean plan time (s)
find-and-tray-stack	**semantic**	**75.6 %**	**124.6**
	uniform	34.4 %	692.1
holding	**semantic**	**65.6 %**	**134.3**
	uniform	47.8 %	438.5
stack	**semantic**	**64.4 %**	**17.2**
	uniform	35.6 %	924.1

Same planner, same cell size: the adaptive partition solves $\approx$1.4–2.2$\times$ more scenes, while planning several-fold to $\sim$50$\times$ faster.

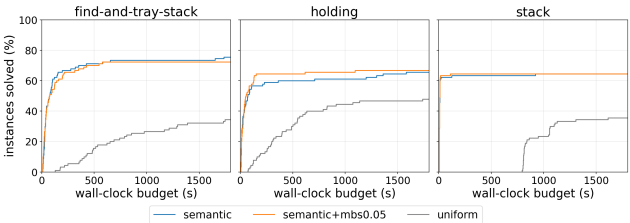
Results: The Cost Gap



`semantic+mbs0.05`: the same partition with a 5 cm minimum free-cell size (a resolution check) — it overlaps `semantic`.

- The adaptive partition **dominates at every budget**; `uniform` starts later and plateaus lower

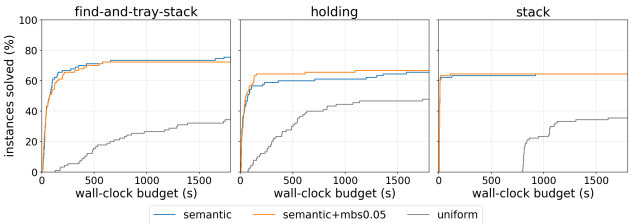
Results: The Cost Gap



`semantic+mbs0.05`: the same partition with a 5 cm minimum free-cell size (a resolution check) — it overlaps `semantic`.

- The adaptive partition **dominates at every budget**; `uniform` starts later and plateaus lower
- **Mechanism:** ≈22–38 Boxels vs ∼300 free cells ⇒ ∼300 vs 11k–14k grounded facts

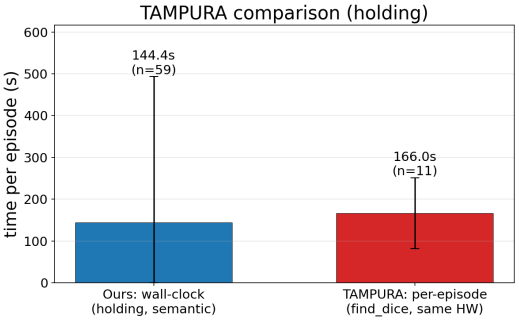
Results: The Cost Gap



`semantic+mbs0.05`: the same partition with a 5 cm minimum free-cell size (a resolution check) — it overlaps `semantic`.

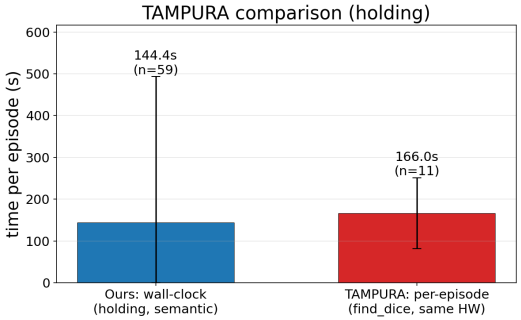
- The adaptive partition **dominates at every budget**; `uniform` starts later and plateaus lower
- **Mechanism:** ≈22–38 Boxels vs ∼300 free cells ⇒ ∼300 vs 11k–14k grounded facts
- Fewer cells ⇒ faster grounding ⇒ more scenes finish in time

Results: Comparison with TAMPURA



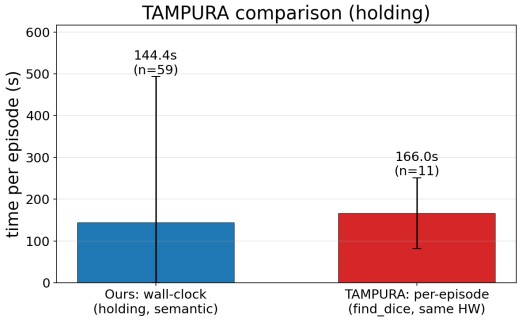
- Rough **parity** end-to-end ($\approx$144 s vs 166 s per episode) and comparable reliability (65.6 % vs the $\geq$63 % its returns imply)

Results: Comparison with TAMPURA



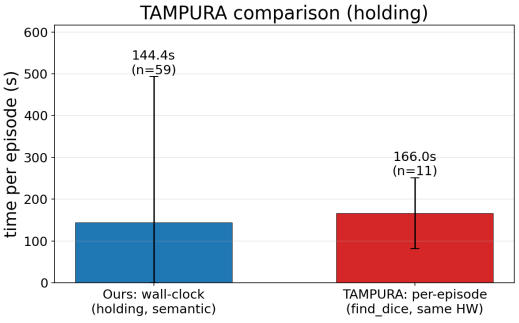
- Rough **parity** end-to-end ($\approx$144 s vs 166 s per episode) and comparable reliability (65.6 % vs the $\geq$63 % its returns imply)
- The contrast is *architectural*: deterministic knowledge-literal planning + replanning vs a **learned probabilistic MDP** solved with LAO*

Results: Comparison with TAMPURA



- Rough **parity** end-to-end (≈144 s vs 166 s per episode) and comparable reliability (65.6 % vs the ≥63 % its returns imply)
- The contrast is *architectural*: deterministic knowledge-literal planning + replanning vs a **learned probabilistic MDP** solved with LAO*
- We hold occlusion as **symbolic state** — line of sight is a planning condition; TAMPURA keeps it sub-symbolic

Results: Comparison with TAMPURA



- Rough **parity** end-to-end (≈144 s vs 166 s per episode) and comparable reliability (65.6 % vs the ≥63 % its returns imply)
- The contrast is *architectural*: deterministic knowledge-literal planning + replanning vs a **learned probabilistic MDP** solved with LAO*
- We hold occlusion as **symbolic state** — line of sight is a planning condition; TAMPURA keeps it sub-symbolic
- Not a benchmark win: different environments, same hardware

Discussion: Limitations

- **Oracle perception:** a ground-truth pose detector, no learned module and no sensor noise — deliberate, to isolate the representation

Discussion: Limitations

- **Oracle perception:** a ground-truth pose detector, no learned module and no sensor noise — deliberate, to isolate the representation
- **One fixed camera:** the robot never reasons about *where to look*

Discussion: Limitations

- **Oracle perception:** a ground-truth pose detector, no learned module and no sensor noise — deliberate, to isolate the representation
- **One fixed camera:** the robot never reasons about *where to look*
- **Simplified manipulation**: top-down grasps, welded grip, a hardcoded post-place lift

Discussion: Limitations

- **Oracle perception:** a ground-truth pose detector, no learned module and no sensor noise — deliberate, to isolate the representation
- **One fixed camera:** the robot never reasons about *where to look*
- **Simplified manipulation**: top-down grasps, welded grip, a hardcoded post-place lift
- **Box-shaped shadows under-cover the hidden region:** the volume an object occludes is really a cone, so a target tucked into the uncovered part gets no Boxel to sense — our most common failure

Discussion: Limitations

- **Oracle perception:** a ground-truth pose detector, no learned module and no sensor noise — deliberate, to isolate the representation
- **One fixed camera:** the robot never reasons about *where to look*
- **Simplified manipulation**: top-down grasps, welded grip, a hardcoded post-place lift
- **Box-shaped shadows under-cover the hidden region:** the volume an object occludes is really a cone, so a target tucked into the uncovered part gets no Boxel to sense — our most common failure
- **Bounded give-up:** on a permanently blocked shadow the planner eventually stops searching — unsound, since the target may still be reachable (we determinize-and-replan, not full contingent POD)

Discussion: Future Work & Takeaway

- **Future work:**

Discussion: Future Work & Takeaway

- **Future work:**
  - a real-robot Franka study

Discussion: Future Work & Takeaway

- **Future work:**
  - 🤖 a real-robot Franka study
  - 🧠 a learned detector on the RGB-D images, dropped in behind the same detection interface

Discussion: Future Work & Takeaway

- **Future work:**
  - 🤖 a real-robot Franka study
  - 🧠 a learned detector on the RGB-D images, dropped in behind the same detection interface
  - 📷 active sensing with a robot-mounted camera (a where-to-look stream)

Discussion: Future Work & Takeaway

- **Future work:**
  - a real-robot Franka study
  - a learned detector on the RGB-D images, dropped in behind the same detection interface
  - active sensing with a robot-mounted camera (a where-to-look stream)
  - recurse into concave geometry (containers, cupboards)

Discussion: Future Work & Takeaway

- **Future work:**
  - 🤖 a real-robot Franka study
  - 🧠 a learned detector on the RGB-D images, dropped in behind the same detection interface
  - 📷 active sensing with a robot-mounted camera (a where-to-look stream)
  - 📦 recurse into concave geometry (containers, cupboards)
  - 🧩 a full semantic free-space merge

Discussion: Future Work & Takeaway

- **Future work:**
  - a real-robot Franka study
  - a learned detector on the RGB-D images, dropped in behind the same detection interface
  - active sensing with a robot-mounted camera (a where-to-look stream)
  - recurse into concave geometry (containers, cupboards)
  - a full semantic free-space merge

Takeaway: making occlusion an explicit, named piece of symbolic state lets a simple *deterministic* planner do partially observable TAMP — reasoning about where to look — without learning a probabilistic policy.

Summary

- **Problem:** TAMP under occlusion — the robot must decide *where to look*, but a dense voxel belief is too large to plan over
- **Idea: Boxels** — task-relevant box-shaped cells (object / shadow / free) that put the structure of space into the symbolic state
- **Method:** a deterministic POD-TAMP planner reasons over Boxels with Know-If fluents; optimistic sensing + replanning resolve the unknown
- **Result:** an order of magnitude fewer cells $\Rightarrow$ far more cluttered scenes solved than a uniform grid; at parity with TAMPURA, without a learned probabilistic policy

Thank you! Questions?

References

[Bon+11] Bonet, B. & Geffner, H. *Planning under Partial Observability by Classical Replanning.* IJCAI, 2011.

[Cou+21] Coumans, E. & Bai, Y. *PyBullet: a Python Module for Physics Simulation for Games, Robotics and Machine Learning.* 2021.

[Cur+24] Curtis, A. et al. *Partially Observable Task and Motion Planning with Uncertainty and Risk Awareness.* arXiv:2403.10454, 2024.

[Gar+18] Garrett, C. R., Lozano-Pérez, T. & Kaelbling, L. P. *PDDLStream: Integrating Symbolic Planners and Blackbox Samplers.* arXiv:1802.08705; ICAPS, 2020.

[Gar+20] Garrett, C. R. et al. *Online Replanning in Belief Space for Partially Observable Task and Motion Problems.* ICRA, 2020.

[Gef+13] Geffner, H. & Bonet, B. *A Concise Introduction to Models and Methods for Automated Planning.* Morgan & Claypool, 2013.

[Hor+13] Hornung, A. et al. *OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees.* Autonomous Robots, 2013.

[Kae+98] Kaelbling, L. P., Littman, M. L. & Cassandra, A. R. *Planning and Acting in Partially Observable Stochastic Domains.* Artificial Intelligence, 1998.

[Kuf+00] Kuffner, J. J. & LaValle, S. M. *RRT-Connect: An Efficient Approach to Single-Query Path Planning.* ICRA, 2000.

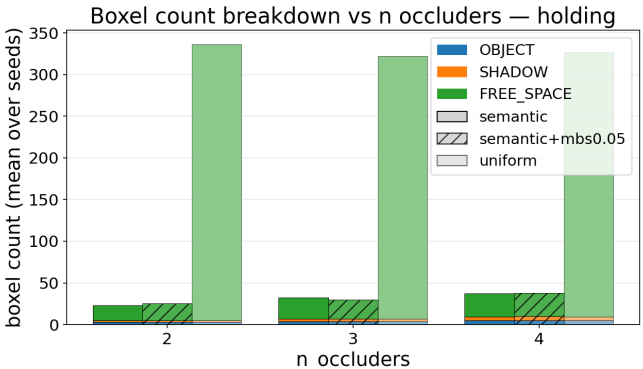
[Mol+20] Molina, D., Kumar, K. & Srivastava, S. *Learn and Link: Learning Critical Regions for Efficient Planning.* ICRA, 2020.

[Pan+24] Pan, T., Shome, R. & Kavraki, L. E. *Task and Motion Planning for Execution in the Real.* IEEE Trans. Robotics, 2024.

[Pet+02] Petrick, R. P. A. & Bacchus, F. *A Knowledge-Based Approach to Planning with Incomplete Information and Sensing.* AIPS, 2002.

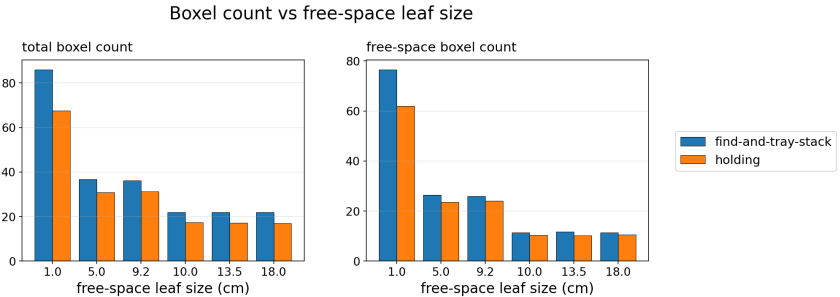
[Zha+25] Zhao, L. et al. *Seeing is Believing: Belief-Space Planning with Foundation Models as Uncertainty Estimators.* arXiv:2504.03245, 2025.

Backup: Representation Compactness



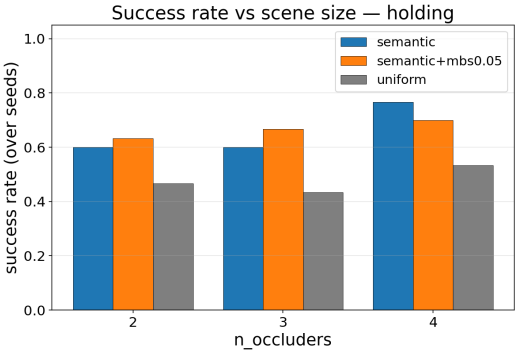
- Adaptive: ≈22–38 Boxels in total
- `uniform`: ∼300 + free-space cells alone
- An order of magnitude fewer cells to ground and search

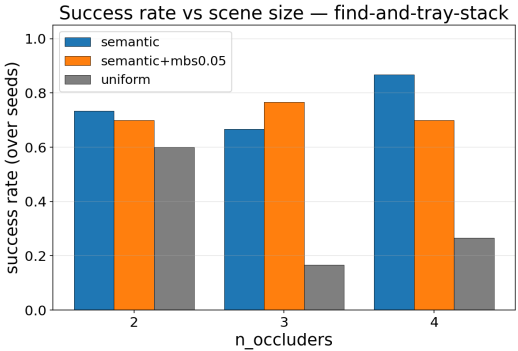
Backup: Free-Space Resolution



The partition is **insensitive** to the free-space leaf size: flat from 5 cm to the auto setting; only the extreme 1 mm floor breaks feasibility (too fine to plan within budget).

Backup: Scaling with Clutter

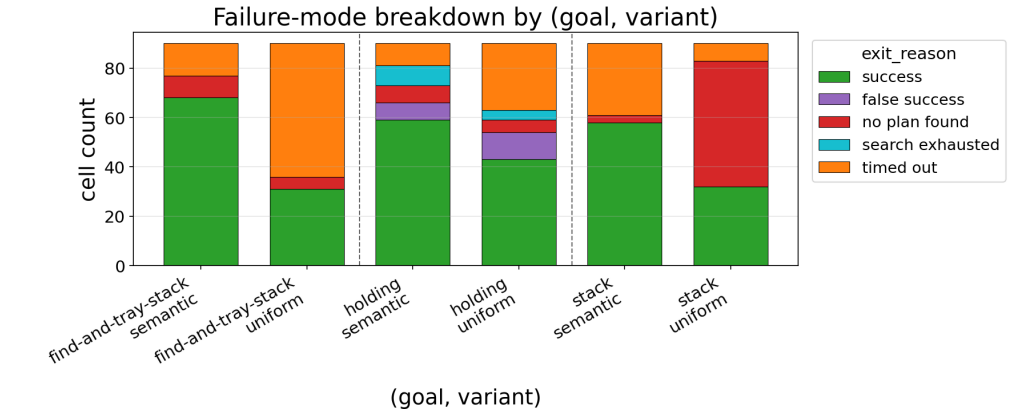




More occluders do not break the adaptive partition: it stays well above `uniform` at every clutter level, and even rises at the densest scenes.

Orange is `semantic+mbs0.05` — the same partition with a 5 cm floor on free-cell size (a resolution check); it tracks `semantic`. `stack` instead scales with tower *height*, not clutter — a manipulation limit, not the planner.

Backup: Failure Modes



Exit reason per (goal, variant), 90 runs each. **uniform** fails mostly by *timing out* (and *no plan found* on `stack`): it cannot ground and search ∼300 cells within budget. **semantic**'s residual failures are fewer, split across *timed out*, *search exhausted* (target outside every box-shaped shadow), and *false success* (a physics mismatch at execution).